



Deep Learning Final Project Report

Semi-Supervised Anomaly Detection for Credit Card Fraud using a β -Variational Autoencoder

By

Chayakarn Hengsuwan

Atikarn Kruaykriangkrai

**Department of Computer Engineering,
Faculty of Engineering,
Kasetsart University**

Academic Year 2026

Table of contents

Chapter 1: Final Project Topic & Motivation.....	1
1.1. Why is this topic interesting?.....	1
1.2. Why Deep Learning?.....	1
Chapter 2: Deep Learning Architecture.....	5
2.1. Model Description.....	5
2.2. Mathematical Formulation.....	6
2.3. Architecture Diagram.....	8
Chapter 3: Code Explanation & Implementation Details.....	10
3.1. Code Explanation.....	10
3.2. GitHub Repository Link.....	15
Chapter 4: Training Method & Dataset.....	16
4.1. Dataset Details.....	16
4.2. Pre-processing.....	17
4.3. Training Hyperparameters.....	22
Chapter 5: Evaluation & Results.....	25
5.1. Training vs. Validation Loss.....	25
5.2. Metrics on Test Set.....	26
5.3. Anomaly Score Analysis.....	31
5.4. Feature Reconstruction Error.....	32
5.5. Latent Space Visualisation.....	33
5.6. Discussion.....	34
Chapter 6: Reference Articles & Related Work.....	35
Chapter 7: Team Contributions.....	36

Chapter 1: Final Project Topic & Motivation

1.1. Why is this topic interesting?

Credit card fraud detection is interesting as a deep learning problem because the difficulty is built into the data structure itself. The dataset contains 284,807 transactions from European cardholders, of which only 492 are fraudulent (0.17%). A model that simply predicts "normal" for every input achieves 99.83% accuracy while catching zero fraud, which means standard classification approaches fail before they even start.

The more interesting constraint is that fraud labels are rarely available in practice. New fraud patterns emerge faster than they can be labeled, so relying on supervised learning creates a moving target. This project approaches the problem differently: a β -Variational Autoencoder trained only on normal transactions, learning what legitimate behavior looks like in compressed form. Fraud is detected not by separating two classes, but by measuring how poorly the model reconstructs something it was never trained to reproduce.

This framing shifts the problem from classification to anomaly detection, which introduces a different set of questions: how to evaluate a model when positive examples are almost nonexistent, how to set a decision threshold without a clean probability output, and how to make the reconstruction error more sensitive to the dimensions that actually differ between fraud and normal. On that last point, EDA identified features V3, V14, and V17 as having the largest distributional shift between classes, and upweighting those features in the loss function meaningfully improved detection performance.

The combination of extreme class imbalance, semi-supervised training, and threshold engineering makes this a more realistic and technically layered problem than standard classification benchmarks.

1.2. Why Deep Learning?

Comparison with other approaches

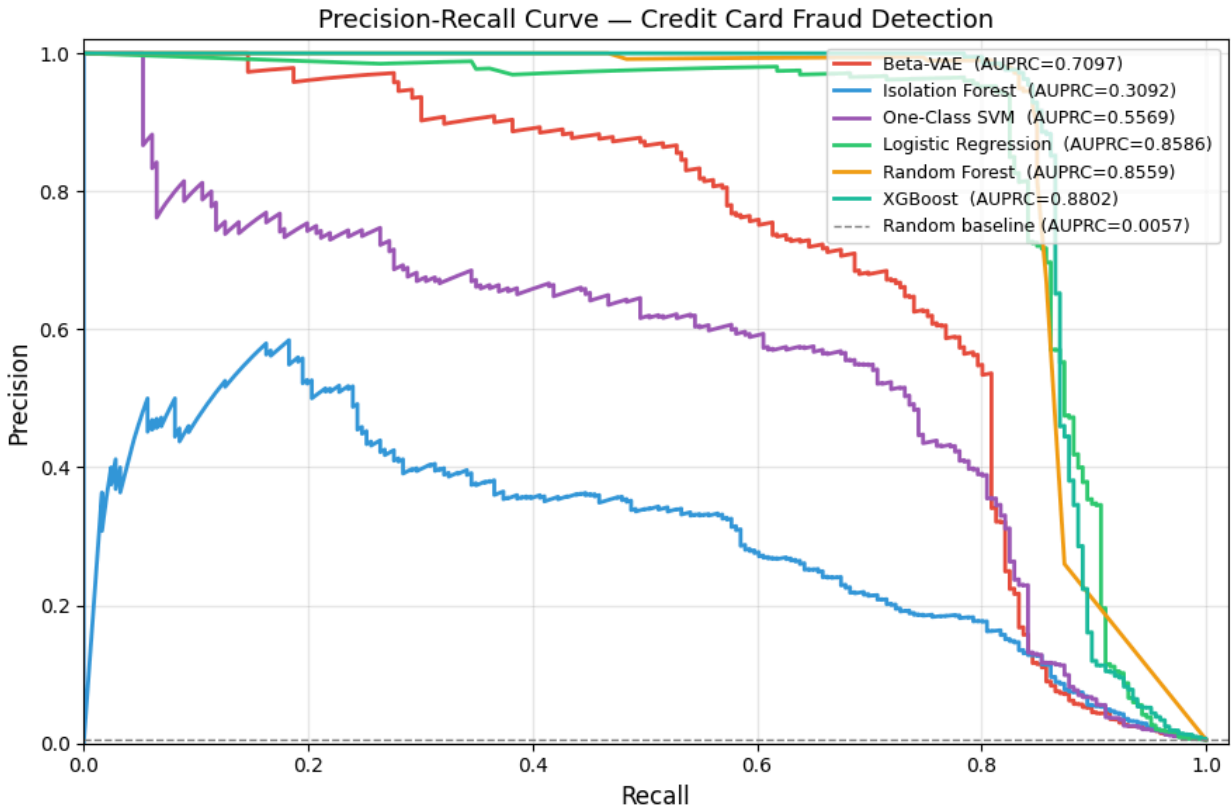
Approach	Strength	Weakness for this problem
Logistic Regression	Fast, interpretable	Cannot model non-linear interactions between PCA components

Approach	Strength	Weakness for this problem
Random Forest / XGBoost	Strong tabular baseline, handles imbalance with class weights	Requires fraud labels at training time; poor calibration under extreme imbalance
Isolation Forest	Unsupervised, no labels needed	Relies on random path length, less effective when fraud is concentrated in correlated PCA subspaces
One-Class SVM	Principled normal-class boundary	Does not scale to 200K training samples; kernel choice is fragile
β -VAE (this project)	Semi-supervised, learns a compact latent manifold of "normal", scales with GPU	Requires tuning β , latent dim, and reconstruction threshold; interpretability limited

Quantitative Comparison on Test Set (sorted by AUPRC)

Supervised models (Logistic Regression, Random Forest, XGBoost) were trained on the combined train and validation sets using fraud labels. Semi-supervised and unsupervised models (β -VAE, One-Class SVM, Isolation Forest) were trained on normal transactions only. All models were evaluated on the same held-out test set.

Model	Fraud Labels?	AUPRC	AUROC	F1	Precision	Recall
XGBoost	Yes	0.8802	0.9781	0.8996	0.9717	0.8374
Logistic Regression	Yes	0.8586	0.9695	0.8788	0.9398	0.8252
Random Forest	Yes	0.8559	0.9359	0.8972	0.9716	0.8333
β -VAE	No	0.7097	0.9607	0.6998	0.6848	0.7154
One-Class SVM	No	0.5569	0.9583	0.6174	0.5661	0.6789
Isolation Forest	No	0.3092	0.9585	0.4193	0.3333	0.5650



Precision-Recall Curve Comparison

Strengths of Deep Learning for this problem (VAE)

1. Semi-supervised by design

The VAE is trained exclusively on the 199,020 normal transactions. It learns a compressed internal representation (latent space) of what a legitimate transaction looks like. It never sees a fraud label during training. At inference time, a fraudulent transaction, which does not conform to the normal data manifold, produces a high reconstruction error. This is the anomaly score. This design means the model remains effective even when fraud labels are completely unavailable for new fraud patterns.

2. Latent manifold compression

The encoder forces the 30-dimensional input through a 4-dimensional bottleneck. The model cannot memorise every input; it must learn the underlying structure. Anomalies that fall outside this learned structure cannot be reconstructed accurately.

3. Feature-weighted loss informed by EDA

The EDA revealed that features V3, V14, and V17 have the largest mean-shift between fraud and normal ($|\Delta\mu| > 6.5$ standard deviations). By upweighting these features in the reconstruction loss (weight = 3.0), the anomaly score becomes more sensitive to the dimensions that actually differ between classes. This is a direct bridge from EDA to architecture.

4. β -regularisation for latent structure

The β term controls the trade-off between reconstruction fidelity and latent space regularity. A small β (0.005) keeps reconstruction quality high, the primary concern for anomaly scoring, while still regularising the latent space enough to prevent overfitting.

Weaknesses/Challenges

The model outputs a reconstruction error, not a probability, so picking a decision threshold is a separate problem with no single correct answer. The threshold chosen depends on what the business cares about: catching more fraud means accepting more false alarms, and finding that balance is ultimately a cost judgment rather than a modeling one.

Detection performance has a ceiling set by the data itself. The AUPRC of 0.676 reflects cases where small-amount fraud and atypical-but-legitimate transactions produce similar reconstruction errors, making them hard to separate regardless of threshold.

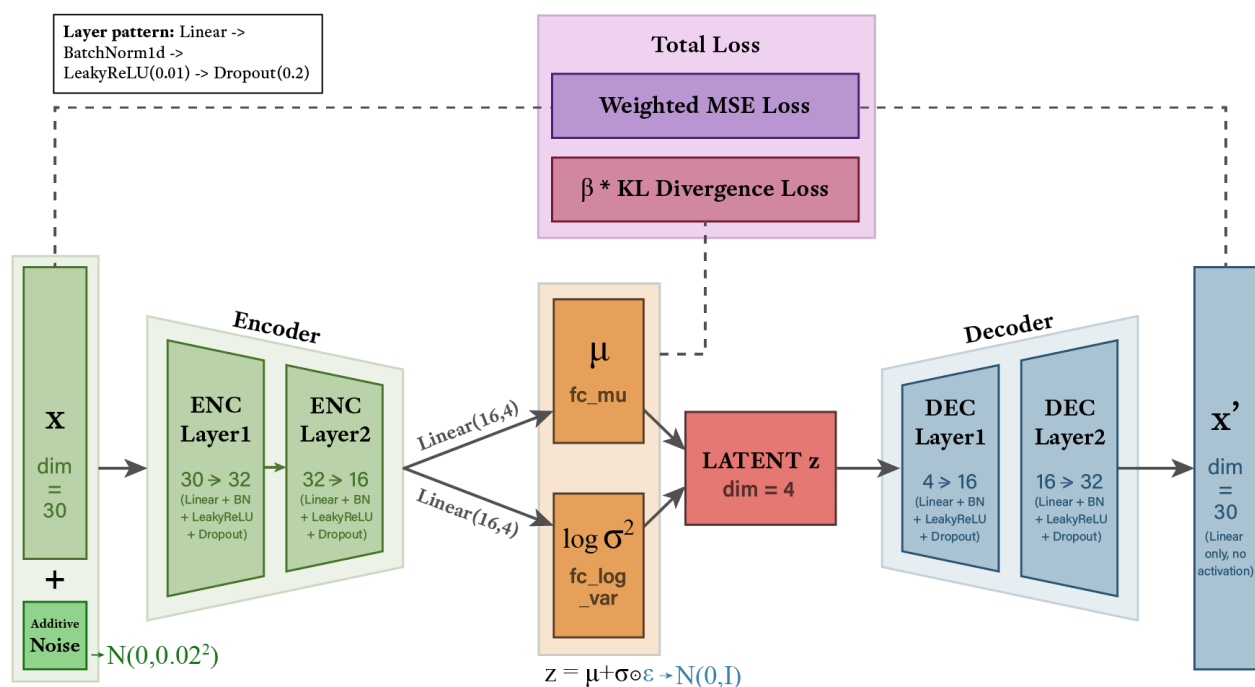
The model also has no mechanism to adapt over time. Fraud patterns shift, and when they do, the model needs to be retrained from scratch on updated normal transaction data.

Chapter 2: Deep Learning Architecture

2.1. Model Description

The model is a **β -Variational Autoencoder (β -VAE)** implemented in PyTorch. It is a generative model composed of three components: an encoder, a latent space parameterisation, and a decoder.

β -VAE Architecture - BetaVAE(nn.Module)



β -Variational Autoencoder (β -VAE) Architecture

Input A 30-dimensional vector representing one credit card transaction, features Time, V1–V28, and Amount, all standardized to $\mu \approx 0$, $\sigma \approx 1$.

Encoder Compresses the input into a lower-dimensional representation through two fully connected layers

LeakyReLU Features V1–V28 are PCA-transformed and contain many negative values. Standard ReLU kills gradients for all negative activations ("dying ReLU"), causing neurons to permanently output zero. LeakyReLU(negative_slope=0.01) passes a small gradient ($0.01 \times x$) for $x < 0$, keeping all neurons alive throughout training.

BatchNorm Each linear layer is followed by BatchNorm, which normalises activations within each mini-batch. This stabilises training by reducing internal covariate shift — especially important here because the feature scale varies significantly across V1–V28, Time, and Amount even after global StandardScaling.

Latent Space Rather than sampling z directly (which is non-differentiable), the model uses:

$$z = \mu + \varepsilon \cdot \sigma \text{ where } \varepsilon \sim N(0, I)$$

During training, ε adds stochasticity that regularises the latent space. During inference (`model.eval()`), the model returns μ directly — producing a deterministic, lower-variance reconstruction error. This determinism is important for a stable anomaly score.

Decoder mirrors the encoder, mapping z back to the 30-dimensional input space.

Why no final activation? The reconstruction target is the StandardScaler-normalised input, which lives on $\mathbb{R}$ (unbounded real values). A sigmoid or tanh activation would clip the output range and introduce systematic reconstruction error. The linear output matches the target space exactly.

Total parameters With this architecture ($30 \rightarrow 32 \rightarrow 16 \rightarrow \mu/\log\sigma^2$ at dim 4, decoder mirrors), the model has approximately **3,400 trainable parameters**, deliberately compact to avoid memorising training samples.

2.2. Mathematical Formulation

The model optimises the **weighted β -Evidence Lower Bound (β -ELBO)**

$$\begin{aligned} L(x) &= E_q[\log p(x|z)] - \beta \cdot KL[q(z|x) || p(z)] \\ &= -recon_loss - \beta \cdot kl_loss \end{aligned}$$

We maximise the ELBO, equivalently minimise

Reconstruction Loss (weighted MSE)

$$recon_loss = (1/N) \cdot \sum_i \sum_d w_d \cdot (x_{\{id\}} - \hat{x}_{\{id\}})^2$$

where w_d is the per-feature weight. Features with high $|\text{mean_fraud} - \text{mean_normal}|$ receive elevated weights (V3, V14, V17 $\rightarrow w=3.0$; V12, V10 $\rightarrow w=2.5$; etc.). Features not listed default to $w=1.0$. This focuses reconstruction pressure on the dimensions most likely to spike when fraud passes through the normal-trained decoder.

KL Divergence (closed form for diagonal Gaussians)

$$kl_loss = (1/N) \cdot \sum_i -\frac{1}{2} \cdot \sum_d (1 + \log \sigma^2_{\{id\}} - \mu^2_{\{id\}} - \sigma^2_{\{id\}})$$

The KL term regularises the posterior $q(z|x) = N(\mu, \sigma^2)$ toward the prior $p(z) = N(0, I)$, preventing the encoder from ignoring the prior and collapsing into a deterministic encoder. Log-variance is clamped to $[-4, 15]$ before exponentiation to prevent numerical overflow.

KL Annealing: β is linearly ramped from $0 \rightarrow 0.005$ over the first 50 epochs

$$\beta_{effective}(epoch) = \min\left(\frac{BETA, BETA \times epoch}{KL_ANNEAL_EPOCHS}\right)$$

This lets the model focus on reconstruction quality first ($\beta \approx 0$), then gradually tighten the latent space. Without annealing, early KL pressure can cause posterior collapse where the encoder ignores the input and outputs the prior, producing uninformative $\mu=0$ for all inputs.

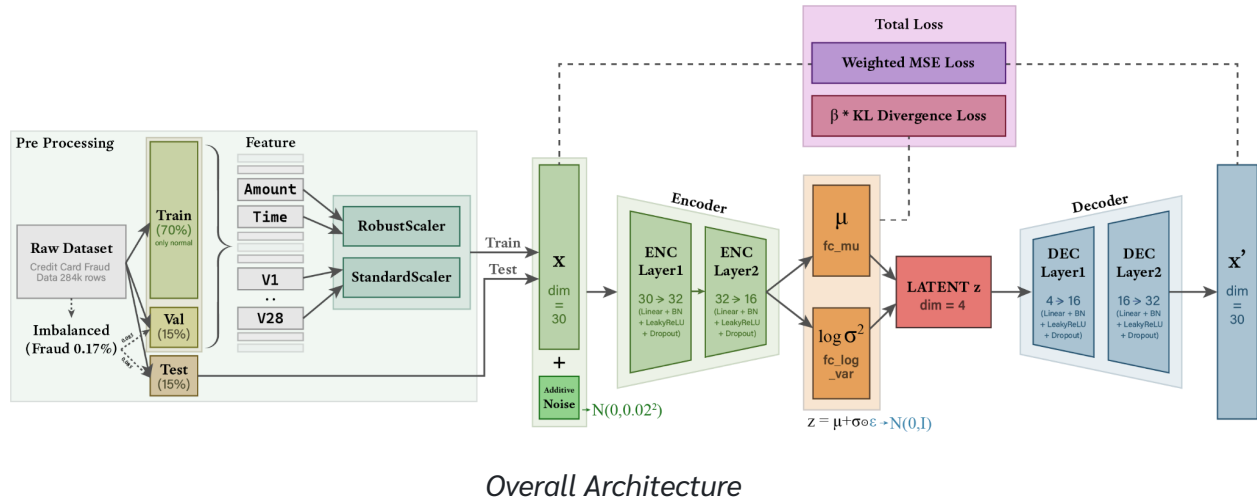
Anomaly Score at inference time

$$score(x) = \left(\frac{1}{D}\right) \cdot \sum_d w_d \cdot (x_d - \hat{x}_d)^2$$

The score is the per-sample weighted mean squared reconstruction error. Higher score $\rightarrow$ more anomalous $\rightarrow$ flagged as potential fraud.

2.3. Architecture Diagram

β -VAE Architecture - Credit Card Fraud Detection



Pre-processing

Splits the dataset into train (70%), validation (15%), and test (15%), with only normal transactions used for training.

Amount and Time are scaled with RobustScaler first. These two features have heavy-tailed distributions: some people spend unusually large amounts or transact at odd hours, which creates extreme outliers. RobustScaler uses the interquartile range instead of mean and std, so those outliers are reduced in scale without being removed entirely.

All 30 features then pass through StandardScaler, bringing everything to mean=0 and std=1. For V1–V28 this is the primary normalisation step, ensuring the PCA components are on a consistent scale before entering the encoder. The result is a standardised 30-dimensional input vector x .

Encoder

Takes x and compresses it through two fully connected layers: 30→32 in the first layer, then 32→16 in the second. Each layer follows the same block structure: Linear → BatchNorm → LeakyReLU → Dropout.

V1–V28 are already PCA-transformed in the dataset, which means many of the values are negative by nature. Standard ReLU would zero out all of those negative activations, effectively discarding a large portion of the input signal. LeakyReLU with a slope of 0.01 fixes this by passing a small gradient for negative values, so the model can still learn from inputs below zero.

BatchNorm is added after each linear layer to stabilize training, and Dropout (0.2) is applied to reduce overfitting.

A small additive noise is also injected into x before encoding during training, which is the denoising component that prevents the model from simply memorising inputs.

The encoder then splits into two parallel heads: fc_mu producing μ and fc_log_var producing $\log \sigma^2$, both at dimension 4.

Latent space

Where the reparameterisation happens. Rather than sampling z directly from the distribution, the model computes $z = \mu + \varepsilon \cdot \sigma$ where ε is drawn from $N(0, I)$. This keeps the operation differentiable during backpropagation. The latent vector z has dimension 4, the bottleneck of the entire model.

Decoder

z is expanded back through two fully connected layers, $4 \rightarrow 16$ then $16 \rightarrow 32$, each following the same block structure as the encoder: Linear $\rightarrow$ BatchNorm $\rightarrow$ LeakyReLU $\rightarrow$ Dropout. A final linear layer then outputs x' of dimension 30 with no activation, matching the unbounded StandardScaler target space.

Loss

The total loss combines two terms. Weighted MSE measures how accurately the decoder reconstructs the input. Rather than treating all 30 features equally, feature weights are assigned based on EDA findings: features where the mean difference between fraud and normal transactions is largest get higher weights. V3, V14, and V17 each received a weight of 3.0 because EDA showed they carry the strongest fraud signal ($|\Delta\mu|$ above 6.5). The full feature weight table is defined in [config.py](#)

Chapter 3: Code Explanation & Implementation Details

3.1. Code Explanation

3.1.1. EDA

01_eda.ipynb ([source code](#))

Uses: src/config.py only

Loads creditcard.csv using `cfg.RAW_CSV` and `cfg.TARGET_COL` from `config`. Everything else runs inline with `pandas`, `matplotlib`, and `seaborn`. In this process, we have done

- Checks shape, dtypes, and class distribution (284,315 normal / 492 fraud)
- Plots feature distributions for V1–V28
- Draws a correlation heatmap
- Identifies which features have the largest mean difference between fraud and normal.
This result is what feeds into `FEATURE_WEIGHTS` in `config.py`

02_post_prep_eda.ipynb ([source code](#))

Uses: `data/processed/*.npy` directly, no `src/` imports at all

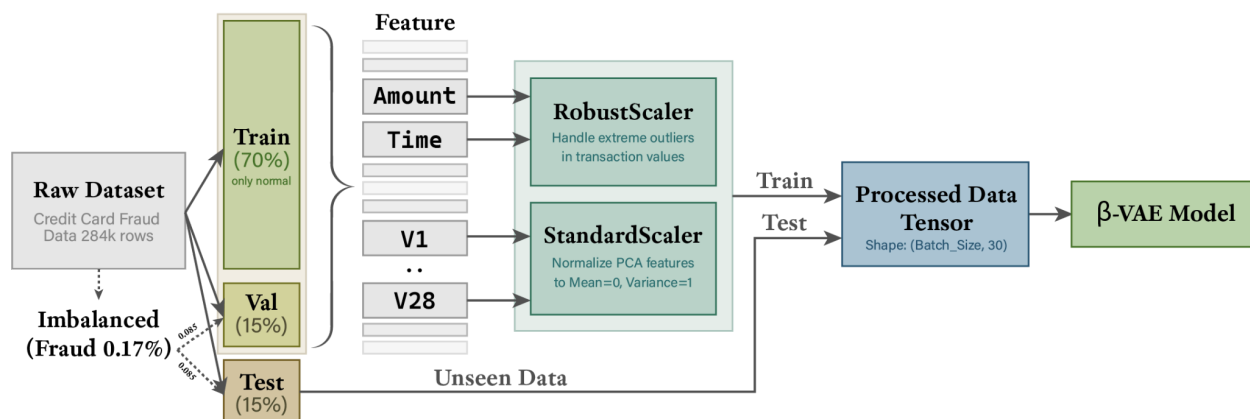
Loads the processed arrays with `np.load()` to verify the preprocessing pipeline worked correctly.

During this work, we have check

- Scaling brought all features to $\mu \approx 0$, $\sigma \approx 1$
- Split sizes and fraud rates match expectations
- PCA feature boxplots for normal vs. fraud on processed data
- No data leakage between train/val/test splits

3.1.2. Pre-process

Preprocessing Architecture - Credit Card Fraud Detection



Pre-processing Architecture

src/preprocess.py ([source code](#))

Uses: src/config.py → for RAW_CSV, TARGET_COL, DATA_PROC_DIR, RANDOM_SEED

Four functions chained by preprocess() which are

Function	What it does
load_data()	Reads creditcard.csv, validates all 30 columns are present
split_semi_supervised()	Normal rows → 70/15/15 train/val/test. Fraud rows → 50/50 val/test only. Training set has zero fraud by design
scale_features()	Stage 1: RobustScaler on Time & Amount. Stage 2: StandardScaler on all 30 features. Stage 3: clip to ± 5.0 . All fitted on X_train only — no leakage
save_artifacts()	Writes X_train.npy, X_val.npy, X_test.npy, label arrays, feature_columns.json, and scaler.pkl to data/processed/

3.1.3. Train

03_train.ipynb ([source code](#))

Uses: src/config.py, src/preprocess.py, src/train.py, src/model.py

Handles the full Colab setup by clones the GitHub repo, installs requirements, sets up Kaggle credentials, and downloads creditcard.csv. Then

1. Calls src.preprocess as a subprocess to produce the processed arrays
2. Mounts Google Drive to persist experiment results across VM resets
3. Patches cfg.EXPERIMENTS_DIR to point to Drive
4. Runs training with a single call: from src.train import train; exp_dir = train()
5. Reads loss_history.csv and plots training curves inline
6. Reloads the best checkpoint and shows an anomaly score histogram on the validation set

src/train.py ([source code](#))

Uses: src/config.py, src/model.py

Function	What it does
load_data()	Wraps X_train in a DataLoader (batch_size=512, shuffle=True, drop_last=True). Keeps full X_val as a plain tensor
_build_feature_weights()	Builds a (30,) float32 tensor from config.FEATURE_WEIGHTS. Features not in the dict default to 1.0
_build_noise_sigma()	Per-feature noise levels, inversely proportional to feature weights — high-weight features get less noise to preserve their fraud signal
_train_epoch()	For each batch: optionally adds per-feature noise to input, runs model(x_in), computes vae_loss(), calls loss.backward(), clips gradients at max_norm=1.0, steps the optimizer
_val_epoch()	Runs full val set through model.eval() (deterministic μ , no noise). Returns val_loss, AUROC, AUPRC

Function	What it does
train()	Main loop: KL annealing ramps β from 0 $\rightarrow$ 0.005 over 50 epochs, early stops after 20 epochs without AUPRC improvement, saves checkpoint on every AUPRC improvement

Why AUPRC instead of val_loss?

With only 0.17% fraud, a model can achieve very low reconstruction loss while completely failing to separate fraud from normal. AUPRC directly measures ranking quality. Each checkpoint stores: model_state, optim_state, val_auprc, val_auroc, input_dim, latent_dim, beta.

src/model.py ([source code](#))

Uses: src/config.py $\rightarrow$ for ENCODER_DIMS, DECODER_DIMS, LATENT_DIM, LEAKY_RELU_SLOPE, BETA

Function	What it does
_build_encoder()	Funnel 30 $\rightarrow$ 32 $\rightarrow$ 16 with LeakyReLU (chosen over ReLU because V1–V28 are PCA-transformed and contain many negatives — ReLU would kill those gradients)
_build_decoder()	Mirror funnel 4 $\rightarrow$ 16 $\rightarrow$ 32 $\rightarrow$ 30. Final layer has no activation — output targets are in StandardScaler space and can be any real value
reparameterise()	Training: returns $\mu + \text{eps} * \text{std}$. Eval: returns μ directly — deterministic output for a stable anomaly score
vae_loss()	Weighted MSE reconstruction loss + KL divergence ($\log_var$ clamped to $[-4, 15]$ for numerical stability). Total = recon_loss + beta * kl_loss

3.1.4. Evaluate

04_evaluate.ipynb ([source code](#))

Uses: src/evaluate.py, src/train.py (_build_feature_weights), src/config.py, src/model.py (via evaluate.py internally)

Imports a set of functions from src/evaluate.py and runs the full threshold analysis pipeline, producing all evaluation plots and saving results to JSON.

src/evaluate.py ([source code](#))

Uses: src/model.py, src/config.py, src/train.py

Function	What it does
compute_anomaly_scores()	Loads checkpoint, runs model.eval(), returns per-sample weighted reconstruction error $(x - \hat{x})^2 \cdot \text{feature_weights}$ averaged over 30 features → (N,) array
threshold_analysis()	Sweeps 500 thresholds, computes TP/FP/FN/TN/Precision/Recall/F1 and total cost at each. Cost matrix: FN (missed fraud) = 1,000 penalty units, FP (wrongly blocked) = 100 penalty units
find_optimal_thresholds()	Returns three cut-offs: max_f1 (best F1), min_cost (lowest total cost (in penalty units)), recall90 (highest threshold still achieving recall ≥ 0.90)
plot_threshold_curves()	3-panel figure: Precision/Recall/F1 vs. threshold · Cost vs. threshold · Full PR curve with AUPRC and AUROC
compute_per_feature_errors()	Returns (N, 30) weighted squared errors per feature per sample
plot_feature_reconstruction_error()	Grouped bar chart of mean error per feature for normal vs. fraud, sorted by fraud–normal gap — shows which features drive the anomaly score
compute_latent_mu()	Encodes dataset, returns (N, latent_dim) posterior mean μ vectors

Function	What it does
<code>plot_latent_tsne()</code>	T-SNE on stratified subsample of μ vectors, coloured by class — shows whether the encoder learned a fraud-discriminative latent space
<code>plot_confusion_matrix_heatmap()</code>	2x2 heatmap annotated with TP/FP/FN/TN counts and their cost (penalty units) at a given threshold
<code>run_evaluation()</code>	End-to-end pipeline: load checkpoint → score → sweep thresholds → find optimal → plot → save thresholds_val.json

3.2. GitHub Repository Link

Link: <https://github.com/Nanokwok/Deep-Fraud-VAE>

Chapter 4: Training Method & Dataset

4.1. Dataset Details

- **Source:** [Credit Card Fraud Detection from Kaggle](#)
- **Size:** 284,807 transactions, 31 columns.
- **Features:**
 1. **Time** - seconds elapsed since the first transaction in the dataset (monotonically increasing over ~48 hours).
 2. **V1-V28** - 28 PCA principal components of the original transaction features. The original features (merchant category, card type, etc.) are not disclosed for confidentiality. The PCA transformation produces features that are already approximately normally distributed, centred near zero, but with varying scales and heavy outlier tails in some components.
 3. **Amount** - transaction amount in euros. Right-skewed: median $\approx$ 22€, max $\approx$ 25,691€.
 4. **Class** - binary label: 0 = Normal, 1 = Fraud. 492 fraud (0.17%).
- **Split used in this project:**

Split	Normal	Fraud	Total	Fraud Rate
Train	199,020	0	199,020	0.00%
Validation	42,647	246	42,893	0.57%
Test	42,648	246	42,894	0.57%

Inherent biases: All transactions are from European cardholders over 2 days in 2013. The model may not generalise to different geographic regions, time periods, or fraud typologies without retraining. Additionally, since V1-V28 are PCA components, the model has no direct access to semantic features (merchant name, card type), which are often the strongest fraud signals in production systems.

See: [reports/eda/figures/class_distribution.png](#) for the class imbalance visualisation and [reports/eda/figures/time_amount_distributions.png](#) for the distribution of Time and Amount.

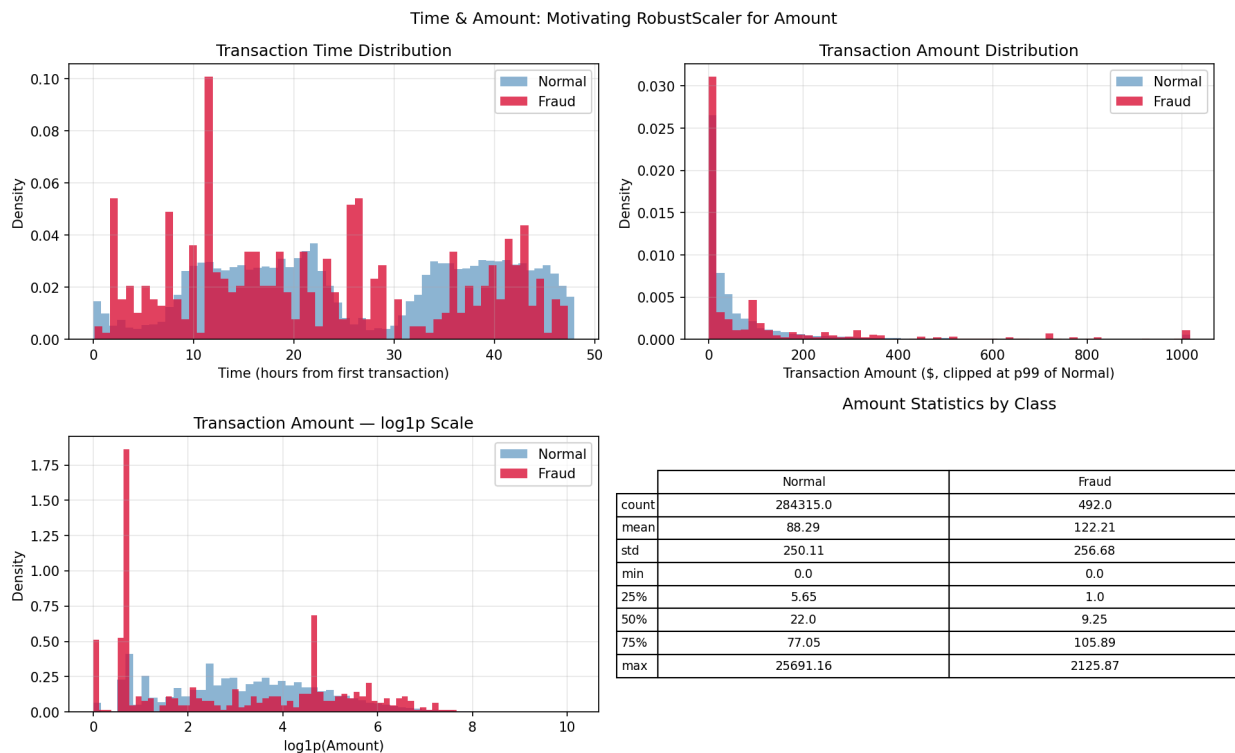
4.2. Pre-processing

Pre-processing is implemented in `src/preprocess.py` ([source code](#)). The pipeline has three stages, all fit exclusively on the training set to prevent data leakage.

Stage 1 RobustScaler on Time and Amount

EDA showed that Amount has a heavy right tail (values up to 25,691€ vs a median of ~22€) and Time is a monotonically increasing counter over 48 hours. Neither of these is centred or unit-scale. RobustScaler uses the interquartile range (IQR) rather than mean/std for centering and scaling. This neutralises the pull of extreme outliers without simply clipping them.

We chose RobustScaler over MinMaxScaler because the outliers carry real information (high-amount transactions can be legitimate or fraud) and we want to reduce their scale, not eliminate them.

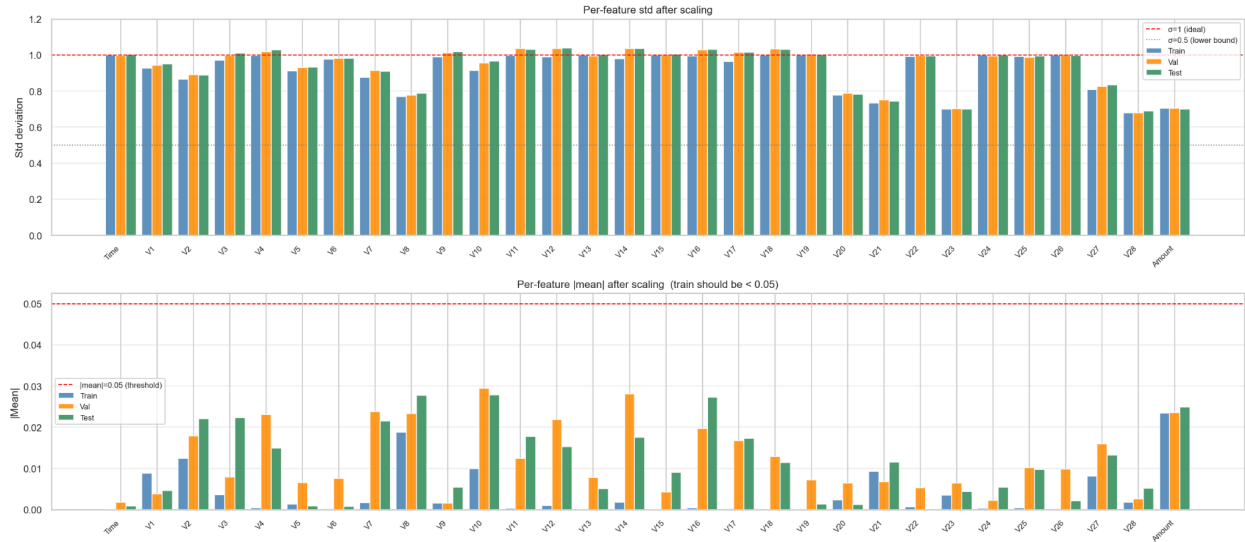


the raw heavy tails that motivated this choice

source: `reports/eda/figures/time_amount_distributions.png` ([click to see](#))

Stage 2 StandardScaler on all 30 features

After RobustScaling Time and Amount, all 30 features are passed through StandardScaler (subtract mean, divide by std) to bring everything to $\mu \approx 0$, $\sigma \approx 1$. This is the target space for MSE-based VAE reconstruction. V1–V28 are already PCA-normalised so this step is near-identity for them, but guarantees consistency and ensures the loss function treats all features comparably before the feature weights are applied.

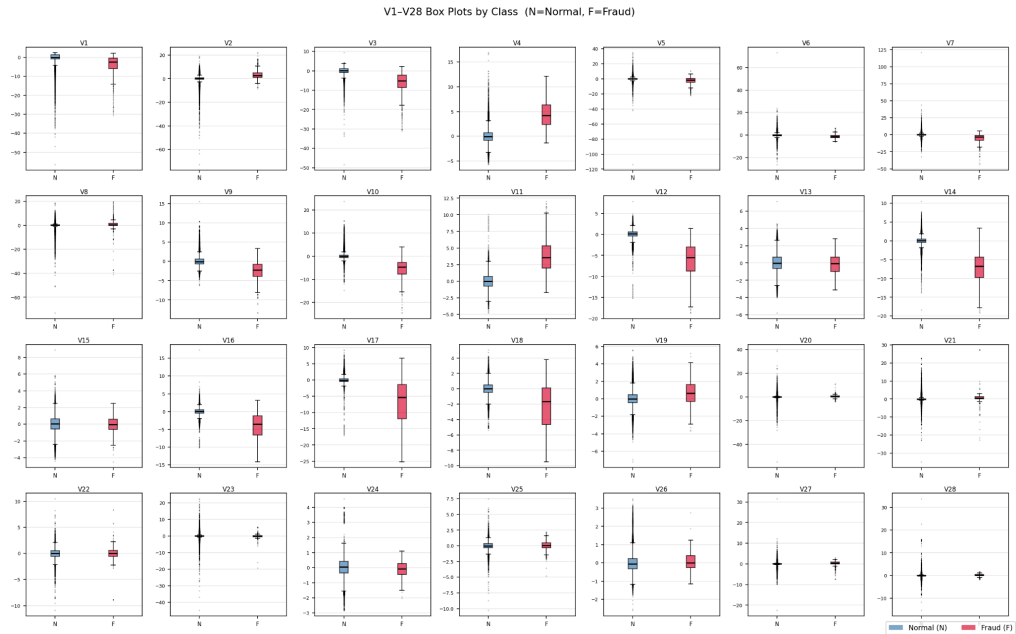


verifies post-scaling distribution consistency across features

source: reports/eda/figures/scaling_quality.png ([click to see](#))

Stage 3 Clip to $\pm 5\sigma$

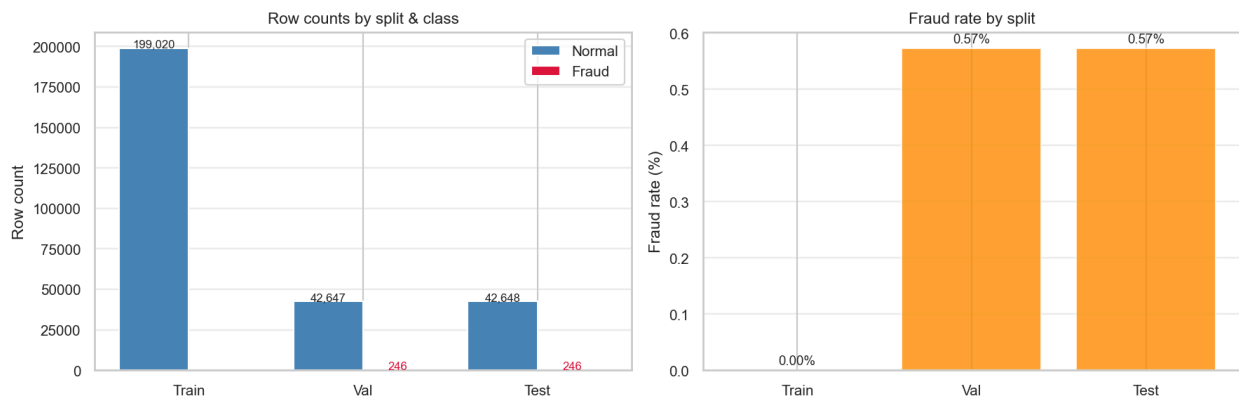
EDA revealed isolated PCA outliers (e.g. V28 exceeds 100σ in a handful of rows). Post-standardisation, these would dominate the MSE loss disproportionately. A $\pm 5\sigma$ clip removes them: in a $N(0,1)$ distribution, fewer than 0.0001% of samples fall outside $\pm 5\sigma$ under normal conditions, so this clip affects only genuine extremes. The same clip is applied identically to all splits (no fitting required).

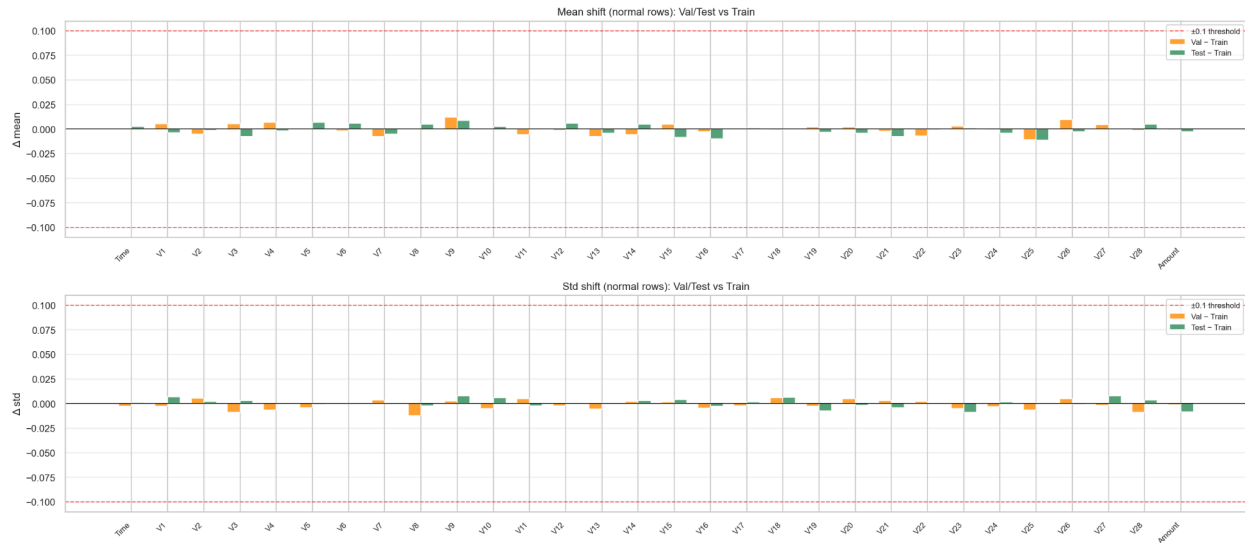


shows the raw PCA outlier structure that necessitated clipping

source: reports/eda/figures/pca_feature_boxplots.png ([click to see](#))

Semi-supervised split strategy: Normal samples are split 70/15/15 into train/val/test. Fraud samples are split 50/50 into val/test (fraud never appears in training). This design ensures: (a) the model never sees a fraud label during training, (b) both val and test sets have representative fraud samples for realistic evaluation, and (c) the validation set is used for early stopping and threshold selection while the test set is held out for final reporting.



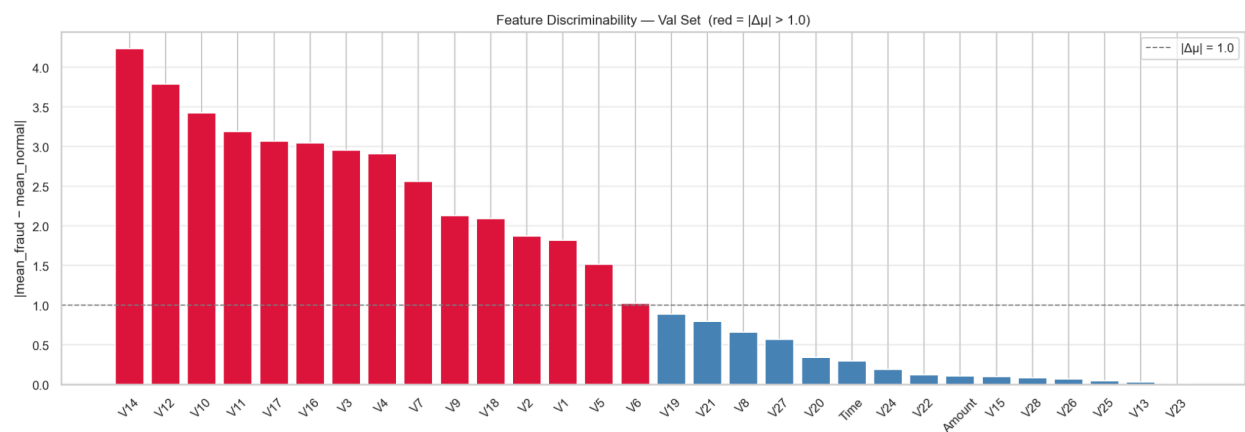


the split sizes and that fraud is absent from training

source: reports/eda/figures/split_summary.png ([click to see](#)) and reports/eda/figures/split_consistency.png ([click to see](#))

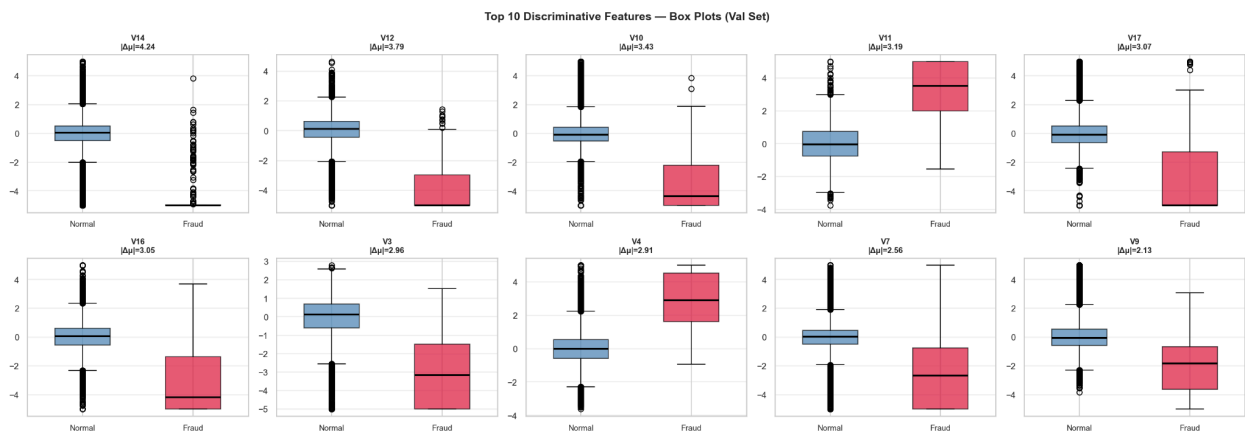
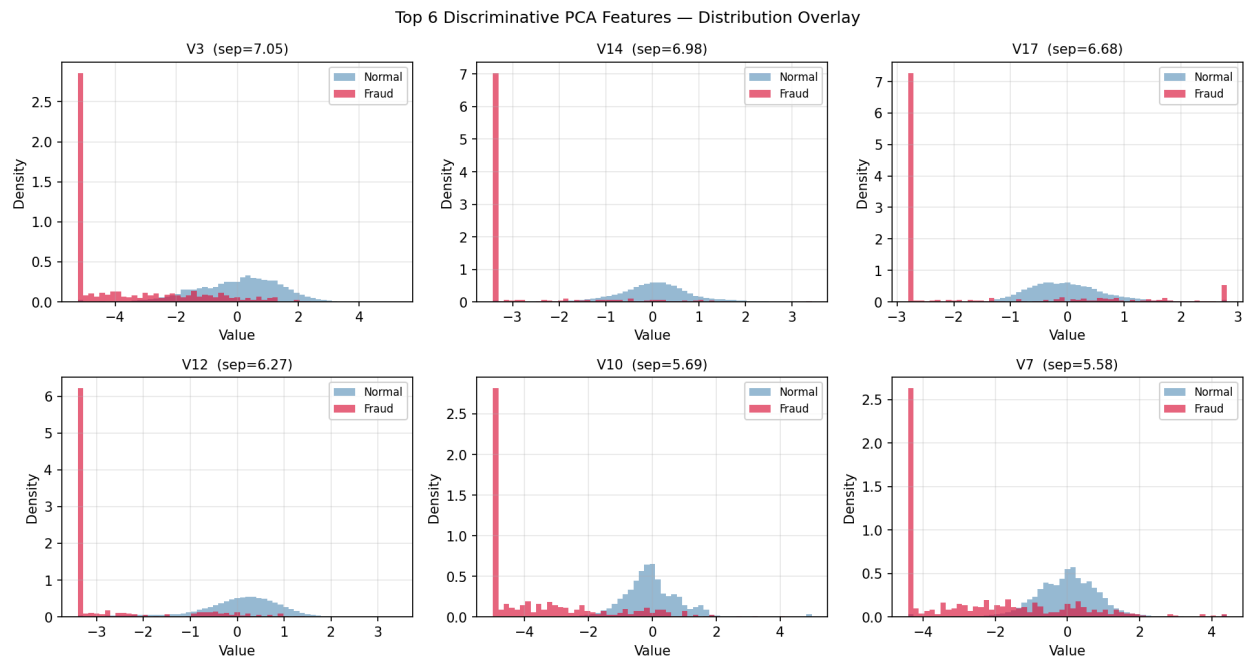
Feature-weighted reconstruction (derived from EDA)

EDA computed the absolute mean difference $|\text{mean_fraud} - \text{mean_normal}|$ for each feature on the full dataset. The top discriminative features were V3 ($|\Delta\mu|=7.05$), V14 (6.98), V17 (6.68), V12 (6.28), V10 (5.68). These features are exactly the ones assigned elevated reconstruction weights in the loss function.



the $|\Delta\mu|$ values per feature that directly drove the `FEATURE_WEIGHTS` config values

source: reports/eda/figures/discriminative_features.png ([click to see](#))



distributional separation between classes for the top features

source: reports/eda/figures/top_features_overlay.png ([click to see](#)) and
reports/eda/figures/top10_boxplots.png ([click to see](#))

4.3. Training Hyperparameters

All hyperparameters are defined in `src/config.py` ([click to see](#)) and backed up per experiment in `experiments/exp_07/config_backup.py`. ([click to see](#))

Hyperparameter	Value	Rationale
INPUT_DIM	30	Time + V1–V28 + Amount
LATENT_DIM	4	Bottleneck compression; too small loses reconstruction fidelity, too large allows memorisation
ENCODER_DIMS	[32, 16]	Funnel: 30→32→16→4; gradual compression avoids information loss
DECODER_DIMS	[16, 32]	Symmetric mirror of encoder
BETA	0.005	Small β keeps reconstruction quality dominant; large β pushes toward prior, collapsing anomaly score sensitivity
LEARNING_RATE	1e-3	Standard Adam initial LR for tabular models
BATCH_SIZE	512	Large enough for stable BatchNorm statistics with 199K training samples
NUM_EPOCHS	200	Upper bound; early stopping fires first
PATIENCE	20	Early stopping: stop if Val AUPRC does not improve for 20 epochs

Hyperparameter	Value	Rationale
LR_PATIENCE	7	ReduceLROnPlateau: halve LR after 7 non-improving epochs
KL_ANNEAL_EPOCHS	50	Linear ramp β : $0 \rightarrow 0.005$ over first 50 epochs
NOISE_STD	0.02	Denoising VAE base noise std ($\approx 2\%$ of unit-std feature)
LEAKY_RELU_SLOPE	0.01	Small pass-through gradient for negative activations
Dropout rate	0.2	Applied after each hidden layer in both encoder and decoder
Gradient clip	1.0	max_norm clip on all parameters per batch
Optimiser	Adam	Adaptive LR per parameter; well-suited for sparse gradient problems
LR decay factor	0.5	LR halved on plateau
Random seed	42	Fixed for full reproducibility

Feature weights (from EDA discriminative analysis)

Feature	Weight	$ \Delta\mu $ (fraud–normal)
V3	3.0	7.05
V14	3.0	6.98
V17	3.0	6.68

Feature	Weight	$ \Delta\mu $ (fraud–normal)
V12	2.5	6.28
V10	2.5	5.68
V7	2.0	5.57
V4	2.0	4.54
V16	2.0	4.14
V1	1.5	4.77
V11	2.0	3.80
All others	1.0	-

Chapter 5: Evaluation & Results

5.1. Training vs. Validation Loss

The model was trained for 200 epochs with early stopping. The best checkpoint (Experiment 07) was saved at epoch 12, at which point Val AUPRC = 0.6760 and Val AUROC = 0.9462.

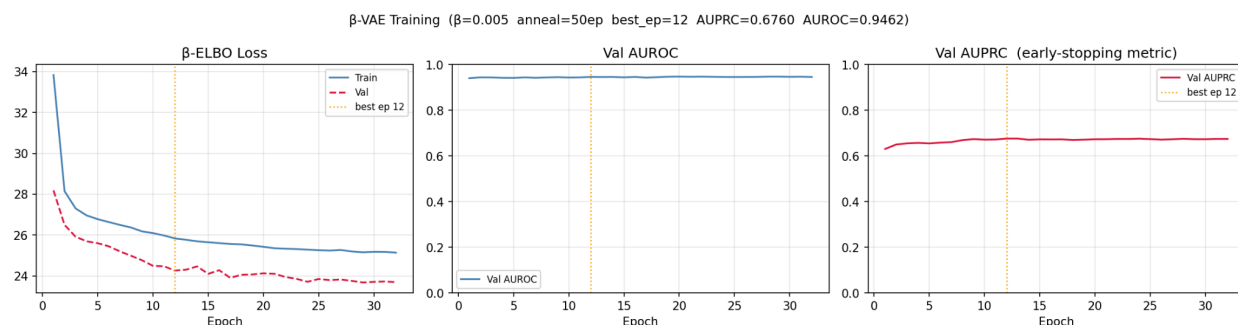
Training loss decreased steadily from 33.83 (epoch 1) to 25.83 (epoch 12). Validation loss followed closely at 24.26, confirming the model did not overfit that training and validation losses tracked together throughout training, with validation loss consistently slightly below training loss (a normal pattern when dropout is applied only during training).

The LR scheduler reduced the learning rate from $1e-3$ to $5e-4$ at epoch 21, and further to $2.5e-4$ at epoch 29, reflecting plateau detection on Val AUPRC. Early stopping fired at epoch 32, 20 epochs after the best Val AUPRC at epoch 12. This indicates that the model found its optimal configuration quickly and subsequent training did not improve fraud detection capability, even as the reconstruction loss continued to decrease slightly. It is worth noting that epoch 12 falls within the KL warm-up phase ($KL_ANNEAL_EPOCHS = 50$), meaning the effective β at the best checkpoint was $0.005 \times (12/50) = 0.0012$, well below the target of 0.005. In practice, Experiment 07 operated closer to a standard autoencoder than a fully regularised β -VAE. This suggests that for this highly imbalanced dataset, reconstruction fidelity is more important to anomaly detection performance than latent space regularisation.

In conclusion, the reconstruction loss continued declining while AUPRC plateaued

Key insight

Lower reconstruction loss $\neq$ better anomaly detection. The model was becoming better at reconstructing all inputs, including fraudulent ones, as it saw more training data. This is exactly why we use early stopping on AUPRC rather than loss.



the three-panel plot (Loss / AUROC / AUPRC over epochs with best-epoch marker)

source: experiments/exp_07/training_curves.png ([click to see](#))

5.2. Metrics on Test Set

The test set contains 42,894 transactions (246 fraud, 42,648 normal, fraud rate 0.57%).

Global discriminability

- Val AUROC: 0.9462

The model ranks a randomly chosen fraud transaction above a randomly chosen normal transaction 94.6% of the time.

- Val AUPRC: 0.6760

Given the extreme imbalance, a random classifier achieves $AUPRC \approx 0.0057$ (baseline = fraud rate). The VAE achieves 0.676, a 118x improvement over random.

Three operating points (test set)

Threshold Strategy	Threshold	Precision	Recall	F1	FP	FN	Cost (Penalty Units)
Max F1	8.696	0.709	0.675	0.692	68	80	86,800
Min Cost	6.252	0.546	0.801	0.649	164	49	65,400
Recall $\geq$ 90%	0.920	0.026	0.931	0.050	8,596	17	876,600

Max F1 (threshold = 8.696)

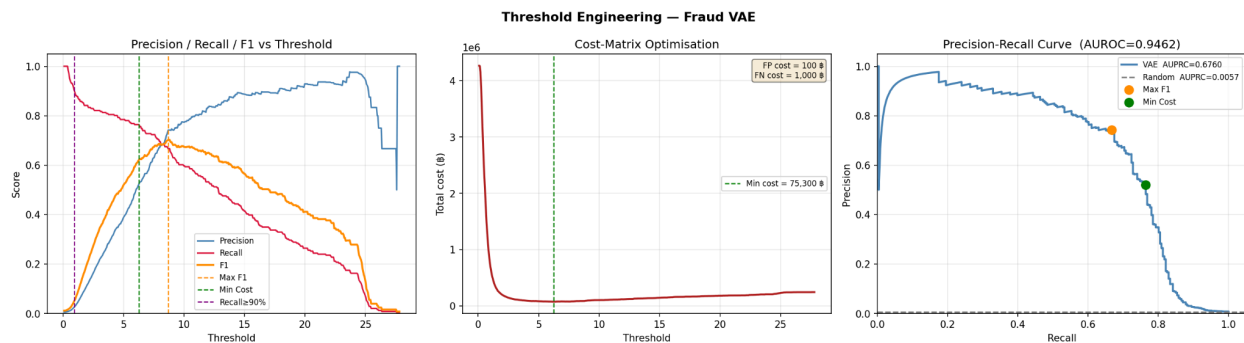
Catches 166 out of 246 frauds (67.5% recall) with 68 false alarms. Precision of 70.9% means 7 in 10 flagged transactions are genuine fraud, acceptable for a human review queue.

Min Cost (threshold = 6.252)

Optimised against a cost matrix where missing a fraud costs 1,000 penalty units and blocking a legitimate customer costs 100 penalty units. Catches 197 out of 246 frauds (80.1% recall) with 164 false alarms. Total cost = 65,400 penalty units per test period, the lowest of the three strategies, making it the operationally preferred setting.

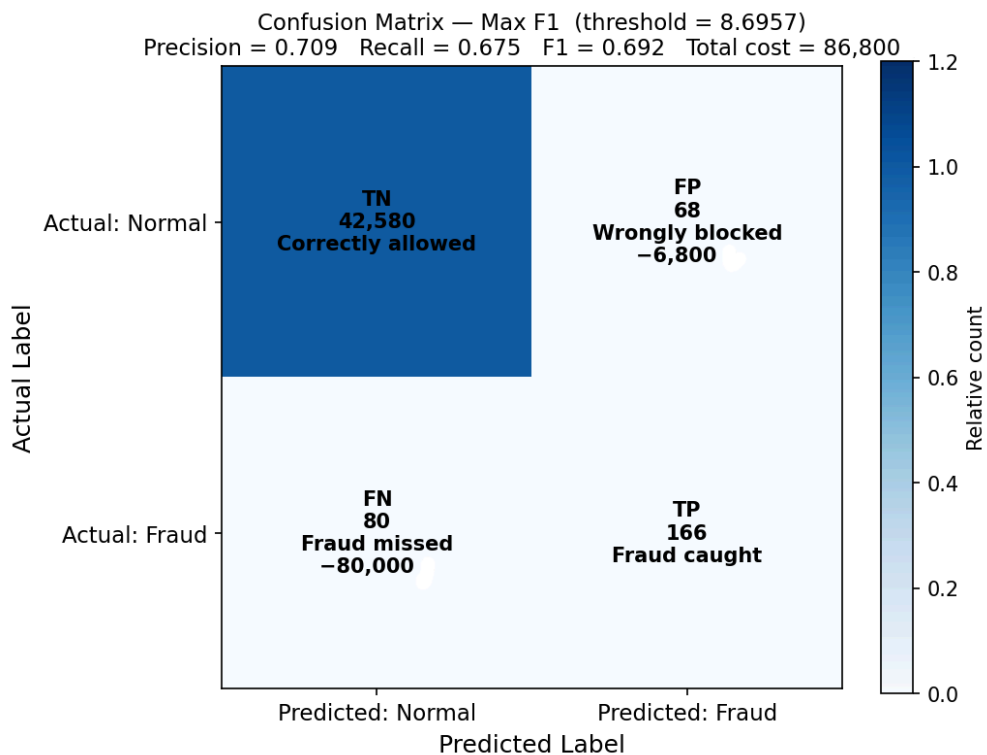
Recall $\geq$ 90% (threshold = 0.920)

Catches 229 out of 246 frauds (93.1% recall) at the cost of blocking 8,596 legitimate transactions. FPR of 20.2% makes this unacceptable for production use without a human review layer.



Precision/Recall/F1 vs threshold, cost curve, and PR curve with operating points marked

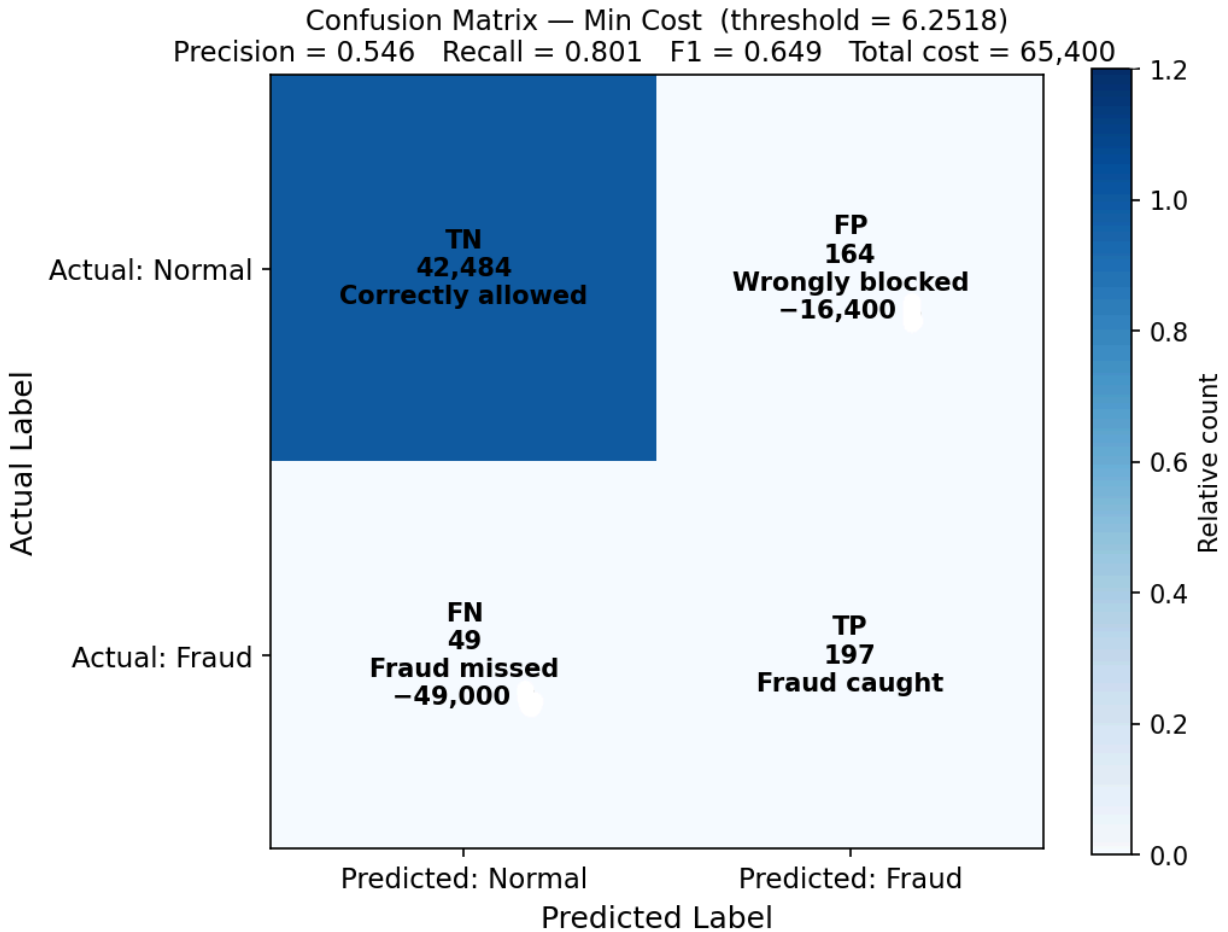
Source: reports/evaluate/figures/threshold_curves_val.png ([click to see](#))



confusion matrix at Max F1 threshold

Source: reports/evaluate/figures/confusion_matrix_max_f1.png [\(click to see\)](#)

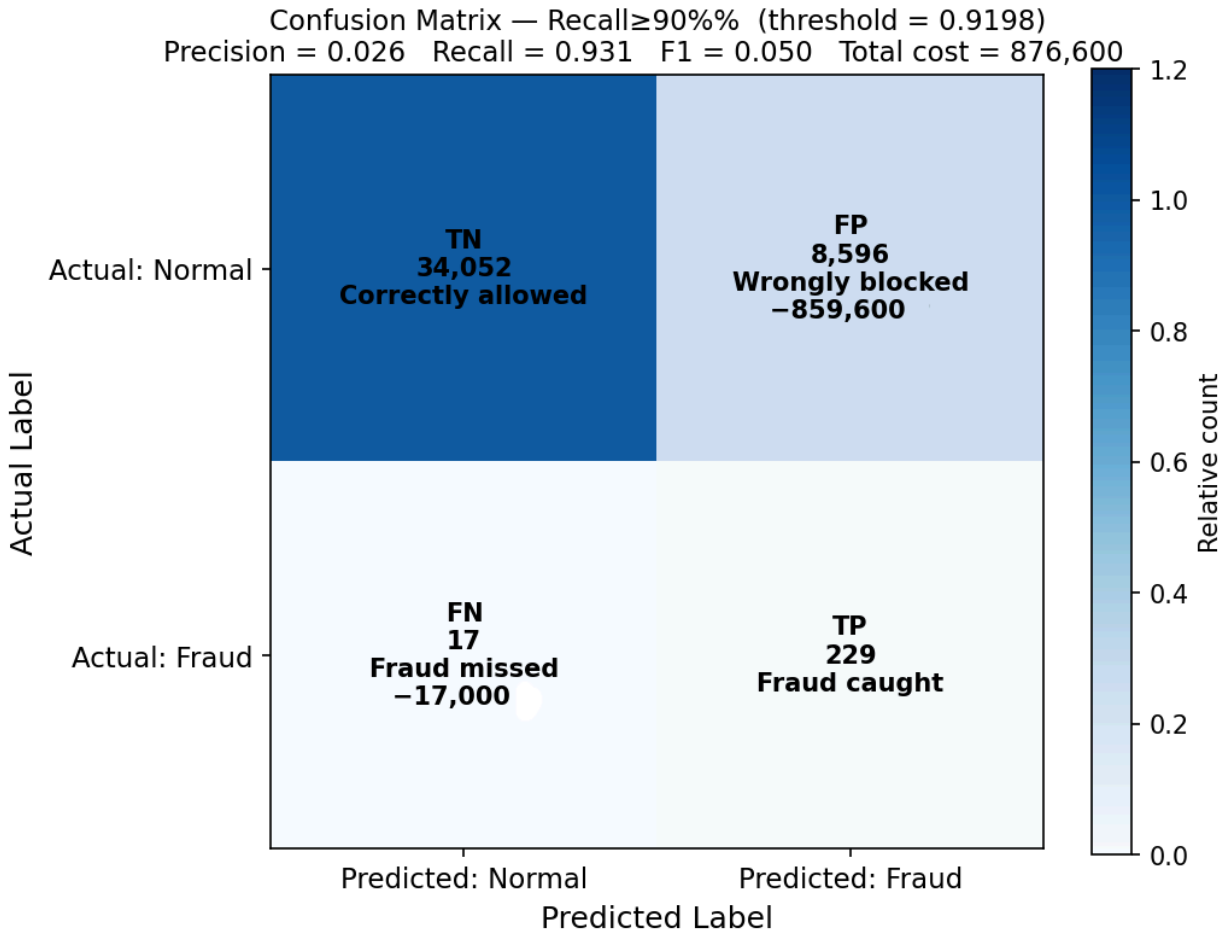
Note: Please ignore the 'B' symbol in the figures, which is a formatting error. The total cost is calculated in 'Penalty Units' based on the dataset's original Euro (€) currency.



confusion matrix at Min Cost threshold

Source: reports/evaluate/figures/confusion_matrix_min_cost.png [\(click to see\)](#)

Note: Please ignore the 'ß' symbol in the figures, which is a formatting error. The total cost is calculated in 'Penalty Units' based on the dataset's original Euro (€) currency.



confusion matrix at Recall $\geq 90\%$ threshold

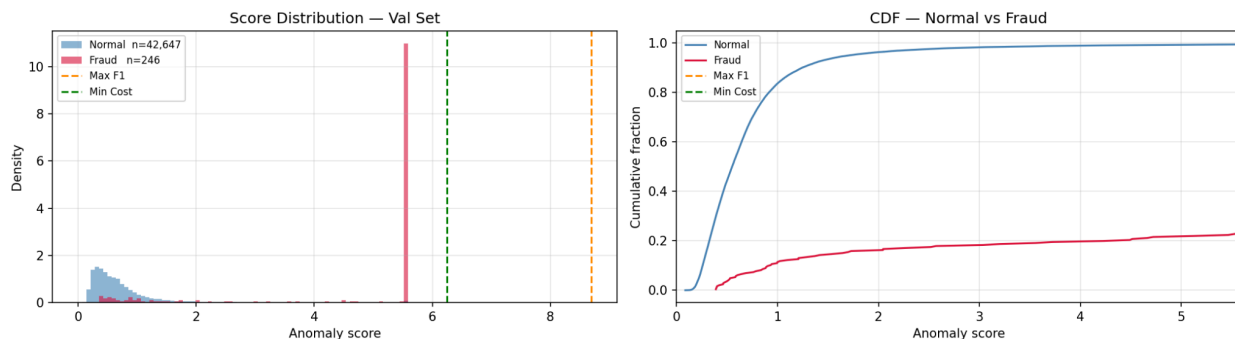
Source: [reports/evaluate/figures/confusion_matrix_recall \$\geq\$ 90pct.png](#) [\(click to see\)](#)

Note: Please ignore the 'ß' symbol in the figures, which is a formatting error. The total cost is calculated in 'Penalty Units' based on the dataset's original Euro (€) currency.

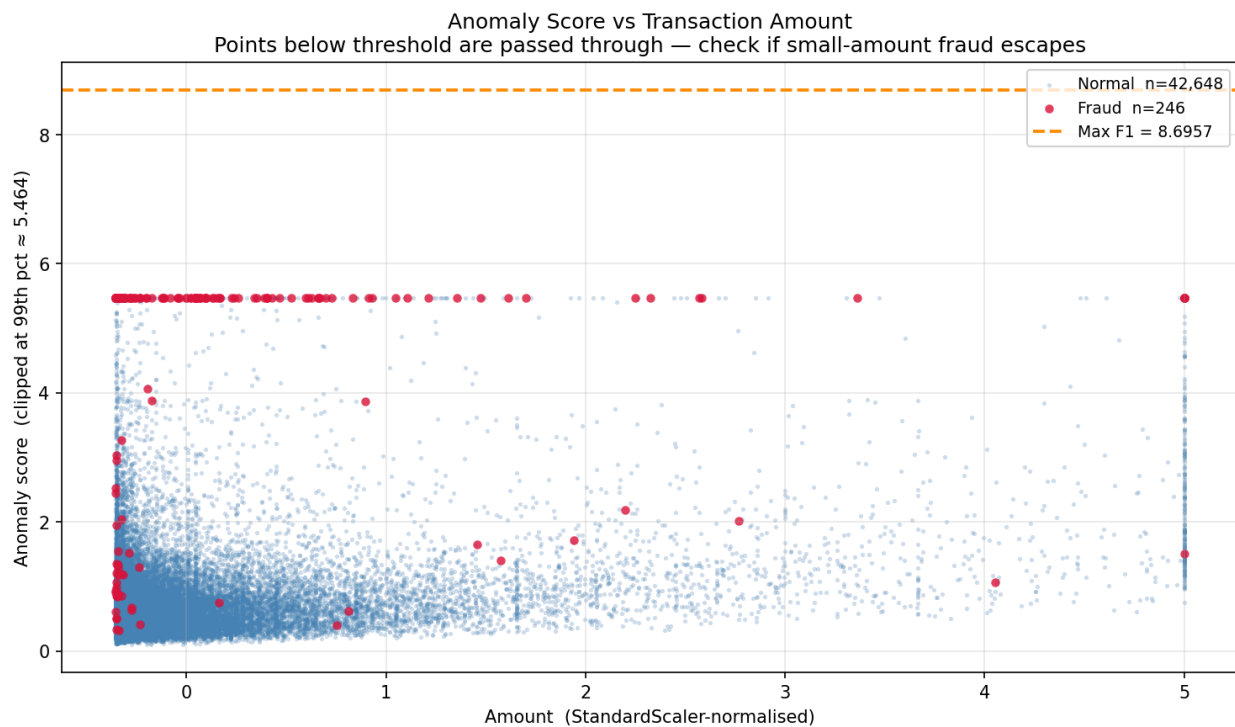
5.3. Anomaly Score Analysis



experiments/exp_07/anomaly_scores.png ([link](#)) shows the reconstruction error histogram for Normal vs Fraud on the validation set. The fraud distribution has a significantly higher mean reconstruction error and a heavier right tail, confirming the VAE has learned a useful normal-class manifold. However, the distributions overlap substantially in the mid-range, which explains why precision is limited: low-value or atypical-normal transactions produce moderate reconstruction errors that overlap with moderate-signal fraud.

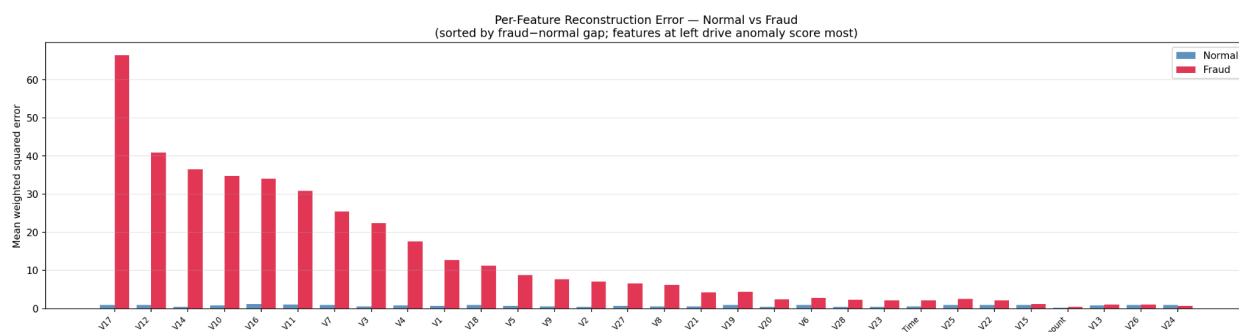


reports/evaluate/figures/score_distribution.png ([link](#)) score distribution on the held-out test set.



reports/evaluate/figures/score_vs_amount.png ([link](#)) scatter of anomaly score vs transaction Amount. This diagnoses amount-bias: frauds of all amounts tend to cluster at higher anomaly scores, but small-amount fraud overlaps with the normal distribution more than large-amount fraud. This is expected: fraudsters often start with small test charges.

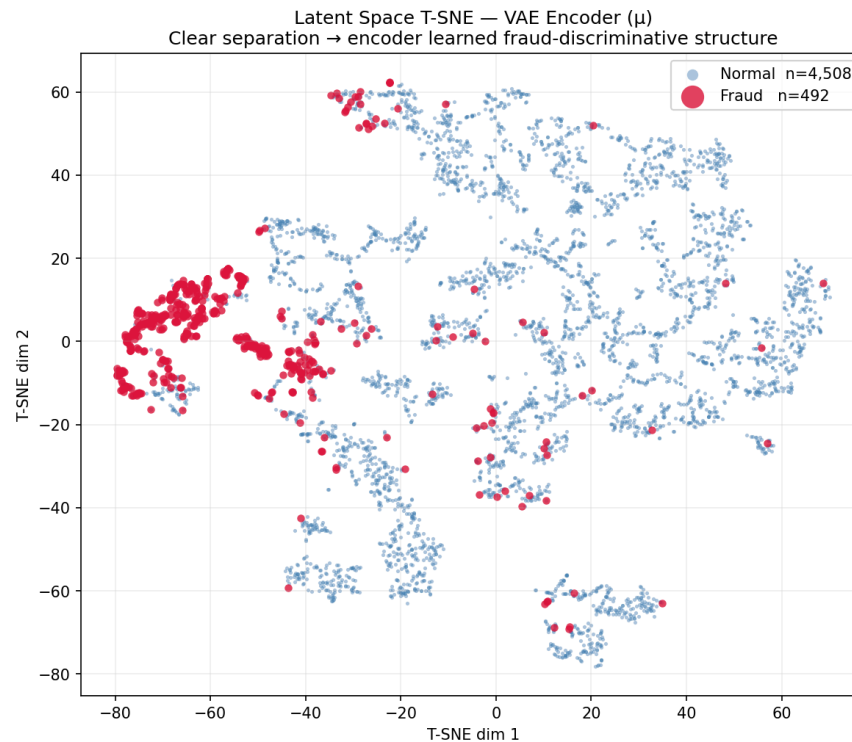
5.4. Feature Reconstruction Error



source: reports/evaluate/figures/feature_reconstruction_error.png ([link](#))

Per-feature mean weighted reconstruction error, sorted by the fraud–normal gap. Features V14, V3, V17, V10, and V12 show the largest gap, confirming that the feature weights assigned from EDA effectively concentrated the anomaly signal in the most discriminative dimensions.

5.5. Latent Space Visualisation



source: reports/evaluate/figures/latent_tsne.png ([link](#))

t-SNE projection of the 4-dimensional latent space (μ vectors) coloured by class. A stratified subsample (up to 5,000 points, balancing class representation) is used so fraud points are visible. If the encoder has learned fraud-discriminative structure despite never seeing fraud labels during training, fraud points will cluster separately from normal points in latent space. Clear cluster separation in this plot would confirm that the model's internal representation is already class-separating, the anomaly score threshold is then just a decision boundary on top of this.

5.6. Discussion

The model performed this way because AUROC of 0.946 shows strong ranking ability where the model clearly separates most fraud from normal. The AUPRC of 0.676 reflects the harder challenge of maintaining high precision at high recall under extreme imbalance. The most common error cases are:

1. False negatives (missed fraud)

Small-amount transactions where the fraud pattern resembles atypical-but-legitimate spending. These frauds use the same PCA-component magnitudes as legitimate transactions, so the VAE reconstructs them reasonably well.

2. False positives (blocked legitimates)

Unusual legitimate transactions (large amounts, unusual timing, rare merchant types mapped to uncommon PCA values) that deviate from the normal training distribution and produce elevated reconstruction errors.

The feature-weighting strategy demonstrably helps: the per-feature reconstruction error plot shows V14, V3, V17 dominating the fraud-normal gap that exactly the features with elevated weights. Without these weights, all 30 features would contribute equally, and the strong signal in V14/V3/V17 would be diluted by noise from the 20 low-signal features

Chapter 6: Reference Articles & Related Work

Diederik P Kingma, Max Welling. (2013). Auto-Encoding Variational Bayes. *arXiv:1312.6114*.
<https://arxiv.org/abs/1312.6114>

Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, Alexander Lerchner. (2017). beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. *ICLR 2017*. <https://openreview.net/forum?id=Sy2fzU9gl>

Jinwon An, Sungzoon Cho. (2015). Variational Autoencoder based Anomaly Detection using Reconstruction Probability. *SNU Data Mining Center Technical Report*.
<https://www.semanticscholar.org/paper/Variational-Autoencoder-based-Anomaly-Detection-An-Cho/061146b1d7938d7a8dae70e3531a00fceb3c78e8>

Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating Probability with Undersampling for Unbalanced Classification. *SSCI 2015*. (Original paper associated with the Kaggle dataset.)

PyTorch Documentation - torch.nn, torch.optim, DataLoader.
<https://docs.pytorch.org/docs/2.11/index.html>

Kaggle Dataset - Credit Card Fraud Detection. Machine Learning Group, ULB.
<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

Chapter 7: Team Contributions

Team Member Name	Specific Tasks Completed	% of Total Work
6510545322 Chayakarn Hengsuwan	• Tuned hyperparameters collaboratively via src/config.py: adjusted dropout rate, beta, epoch count, encoder/decoder dims across multiple config iterations • Built model_comparison.ipynb: compared β-VAE against Isolation Forest, One-Class SVM, Logistic Regression, Random Forest, and XGBoost on the same held-out test set • Fixed Kaggle dataset download and Google Drive integration in the training and comparison notebooks • Wrote the comparison results section of the final report, including the qualitative comparison table (Approach / Strength / Weakness) and the quantitative results table (AUPRC, AUROC, F1, Precision, Recall) • Wrote and documented all explanation sections of the final report	50%
6510545799 Atikarn Kruaykriangkrai	• Set up the project structure and repository • Conducted initial EDA (01_eda.ipynb): class distribution, feature distributions, correlation heatmap, identifying discriminative features (V3, V14, V17, etc.) • Built the data preprocessing pipeline (src/preprocess.py) — semi-supervised 3-way split, RobustScaler + StandardScaler + clipping pipeline, saving processed arrays • Conducted post-preprocessing EDA (02_post_prep_eda.ipynb) — verified scaling quality, split consistency, PCA feature boxplots	50%

Team Member Name	Specific Tasks Completed	% of Total Work
	• Implemented the β-VAE architecture (src/model.py) — encoder/decoder with LeakyReLU, reparameterisation trick, weighted β-ELBO loss • Built the full training loop (src/train.py): KL annealing, feature-specific denoising, AUPRC-based early stopping, checkpointing • Ran 7 training experiments, iteratively tuning: beta, latent dimensions, encoder/decoder dims, LeakyReLU activation, NOISE_STD, KL anneal schedule • Built the evaluation pipeline (src/evaluate.py): threshold sweep, 3 optimal thresholds, cost matrix, PR curve, latent t-SNE, feature reconstruction error plots • Ran full model evaluation (04_evaluate.ipynb) and generated all figures in reports/evaluate/	